Design Decisions — Retail Store Pricing Management System

1. Purpose

This document records the main technical design decisions for the Retail Store Pricing Management System. The application must allow authorised users to upload retail pricing CSV files, search pricing records, and edit and save records. The target scale is approximately 3,000 stores across multiple countries.

2. Decision Summary

Decision 1: Use React for the frontend

Decision: Use a React Single Page Application (SPA).

Rationale: - Supports interactive upload, search, filtering, and edit forms. - Allows reusable UI components for tables, forms, validation messages, and status feedback. - Separates the user interface from backend business logic through API calls.

Alternatives considered: Server-rendered HTML or another frontend framework.

Trade-off: A SPA requires client-side state and API error handling. The backend must still validate every request; frontend validation alone is not sufficient.

Decision 2: Use FastAPI for the backend

Decision: Use FastAPI to expose REST endpoints.

Rationale: - Provides a Python-based API layer for request validation and business logic. - Supports typed request and response models. - Fits CSV processing and integrates with PostgreSQL access libraries. - Generates API documentation from endpoint definitions.

Alternatives considered: Flask, Django, or a Node.js backend.

Trade-off: The project must define consistent API error formats, authentication, logging, and database transaction handling rather than assuming these are automatically solved by the framework.

Decision 3: Use PostgreSQL for persistent storage

Decision: Use PostgreSQL as the system of record for pricing records.

Rationale: - Provides durable relational storage and transaction support. - Supports constraints and indexes for data quality and search performance. - Supports structured queries across store, SKU, product, and date fields. - Provides a clear path for future audit and reporting requirements.

Alternatives considered: A document database or file-based storage.

Trade-off: Schema and migration management are required. The data model must define uniqueness, currency, and date semantics before finalising constraints.

Decision 4: Use CSV as the initial feed format

Decision: Accept CSV files with Store ID, SKU, Product Name, Price, and Date.

Rationale: - Matches the specified input format. - Is easy to generate and inspect using common tools. - Provides a straightforward initial import workflow.

Trade-off: CSV has limited type information and can contain malformed rows, inconsistent date formats, or unexpected delimiters. The backend must validate headers and values, limit file size, and return useful row-level feedback.

Decision 5: Validate data on the backend

Decision: Treat FastAPI validation and database constraints as authoritative.

Rationale: - API requests can be made outside the browser. - Central validation applies consistently to uploads and edits. - Database constraints protect data integrity if multiple requests arrive concurrently.

Trade-off: Some invalid input is only rejected after submission. The frontend should provide early feedback for usability, while the backend remains the final authority.

Decision 6: Use paginated search results

Decision: Return bounded pages of pricing records rather than all matching records at once.

Rationale: - Prevents very large responses from overwhelming the browser or API. - Reduces memory and network use. - Makes the search experience more predictable as data volume grows.

Trade-off: Users navigate results across pages. Pagination parameters and a stable sort order must be defined. The appropriate pagination strategy should be reviewed against measured data volume.

Decision 7: Add indexes based on query patterns

Decision: Index fields used frequently in filters and lookups, after reviewing expected queries and query plans.

Rationale: - Store ID, SKU, and date are likely search criteria. - Suitable indexes can reduce database work for common queries.

Trade-off: Indexes consume storage and add overhead to inserts and updates. Do not index every column by default; confirm choices with representative data and query plans.

Decision 8: Use transactions for writes

Decision: Use database transactions for record updates and for each defined unit of CSV import work.

Rationale: - Helps prevent partially applied changes within a transaction. - Provides consistent success or failure handling. - Supports reliable persistence of pricing data.

Trade-off: A whole-file transaction may be expensive for very large files. The import policy must explicitly choose all-or-nothing behaviour or controlled batch processing, with clear reporting of accepted and rejected rows.

Decision 9: Keep database access behind the API

Decision: The React application must call FastAPI; it must never connect directly to PostgreSQL.

Rationale: - Keeps database credentials out of the browser. - Centralises validation, authorisation, and business rules. - Reduces direct exposure of the database.

Trade-off: The API becomes a critical service and needs health checks, error handling, monitoring, and appropriate capacity.

Decision 10: Design for horizontal API scaling, then validate with tests

Decision: Keep API instances stateless where practical and allow multiple instances if demand requires them.

Rationale: - Supports scaling the API independently of the frontend. - Allows capacity to be adjusted based on observed demand. - Fits a deployment behind a reverse proxy or load balancer.

Trade-off: The number of stores alone does not establish required capacity. Feed sizes, upload frequency, concurrent users, and response-time targets must be measured before sizing infrastructure.

Decision 11: Treat security and auditability as explicit requirements

Decision: Plan for authentication, role-based authorisation, HTTPS, protected secrets, restricted database access, and appropriate logging. Add a pricing-change audit trail if required by business or compliance needs.

Rationale: - Pricing updates can affect business operations. - Access should be limited to authorised users. - Logs and audit history help investigate errors and changes.

Trade-off: Identity integration, roles, retention, and audit requirements need agreement. These capabilities must not be described as implemented until they are present and tested.

Decision 12: Confirm multi-country data rules before finalising the schema

Decision: Confirm currency, date, time-zone, and store identifier rules before production release.

Rationale: - A numeric price alone does not identify its currency. - Date interpretation can vary by feed source and business rules. - Store identifiers and SKU formats may vary by country or retailer.

Trade-off: These decisions require business input. Do not infer currency or date semantics from a sample CSV alone.

3. Cross-Cutting Principles

- **Security:** Enforce authentication and authorisation on the backend; use HTTPS in deployed environments.
- **Data integrity:** Validate input, use appropriate database types and constraints, and apply transactions.
- **Performance:** Use pagination, query-specific indexes, bounded upload processing, and representative load tests.
- **Maintainability:** Keep frontend, API, and database responsibilities separated; document API contracts and schema migrations.
- **Observability:** Capture application errors and operational metrics without logging secrets.
- **Recovery:** Configure database backups and periodically test restoration.

4. Decisions Still Requiring Confirmation

The following are open decisions, not assumed implementation facts:

- Authentication provider and role definitions.
- Price currency representation and whether one store can sell in multiple currencies.
- Exact meaning and format of the Date field.
- Pricing-record uniqueness and duplicate-feed handling.
- Maximum file size, expected rows per feed, and upload frequency.
- Whether imports are all-or-nothing or permit valid rows while rejecting invalid rows.
- Expected concurrent users and response-time targets.

- Availability target, backup retention, and recovery objectives.
- Hosting provider, deployment topology, and monitoring tools.
- Whether audit history and reporting are in scope.

5. Conclusion

React, FastAPI, and PostgreSQL provide a practical separation between presentation, application logic, and persistent data. CSV validation, backend-enforced rules, paginated searches, appropriate indexing, transactions, and explicit security controls are central design choices. Final production decisions should be confirmed against business rules and validated through testing with representative data volumes.